

受注管理仕様書

目次

- 1. 概要
- 2. 受注タイプと優先順位
- 3. テーブル定義
- 4. データフロー
- 5. 優先順位制御ロジック
- 6. CSVインポート仕様
- 7. スケジューリング連携

概要

本システムは、複数の得意先から**内示（FORECAST）と確定受注（FIRM）**を受け取り、優先順位を制御しながら生産計画を立てる受注管理システムです。

主な特徴

- **内示と確定の二元管理:** 予測情報（内示）と正式受注（確定）を区別して管理
- **自動優先順位制御:** 確定受注が来たら、対応する内示を自動的に無効化
- **内示の自動上書き:** 毎日更新される内示を最新版で置き換え
- **複数得意先・複数フォーマット対応:** 得意先ごとに異なるCSVフォーマットに対応
- **ステージングテーブル:** CSV取込時の生データを保持し、トラブルシューティングを容易に

受注タイプと優先順位

受注タイプ（order_type）

order_type	日本語	意味	更新頻度	スケジューリング優先度
FIRM	確定	正式な受注、必ず生産する	適宜	最高（必ず生産計画に含める）
FORECAST	内示	予測情報、変更される可能性が高い	毎日	低（確定がない場合のみ使用）

受注ステータス（status）

status	日本語	スケジューリング	説明
OPEN	オープン	○ 対象	有効な受注
SUPERSEDED	無効	× 除外	確定受注により無効化された内示
CLOSED	クローズ	× 除外	完了した受注

status	日本語	スケーリング	説明
CANCELED	キャンセル	× 除外	キャンセルされた受注

受注番号体系（order_no）

内示（FORECAST）：

FC-**{得意先コード}**-**{タイムスタンプ}**

例：FC-000001-20251204153000

確定（FIRM）：

リーデン：FIRM-**{得意先コード}**-**{発注日YYYYMMDD}**

例：FIRM-000001-20251224

ティエラ：FIRM-**{得意先コード}**-**{送付No}**

例：FIRM-000002-1311

クボタ（堺）：FIRM-000196-SAKAI-**{issue_date}**（issue_date=発行日YYMMDD）

例：FIRM-000196-SAKAI-251211

クボタ（枚方）：FIRM-000196-HIRAKATA-**{issue_date}**（issue_date=発行日YYMMDD）

例：FIRM-000196-HIRAKATA-251211

クボタ（枚方・特殊）：FIRM-000196-HIRAKATA-sp-**{issue_date}**（issue_date=発行日YYMMDD）

例：FIRM-000196-HIRAKATA-sp-251211

※ 発注日・送付Noがない場合はタイムスタンプを使用

例：FIRM-000001-20251204153000

テーブル定義

1. t_order（受注ヘッダ）

受注の基本情報を管理するヘッダテーブル。内示（FORECAST）と確定（FIRM）を区別して管理。

テーブル構造

```
CREATE TABLE t_order (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  customer_id BIGINT NOT NULL,  
  order_no VARCHAR(50) NOT NULL,  
  order_type VARCHAR(20) NOT NULL, -- 'FIRM' or 'FORECAST'  
  version_no VARCHAR(20) DEFAULT 'v1',  
  source_system VARCHAR(50),  
  source_file VARCHAR(200),  
  order_date DATE,  
  freeze_from DATE,
```

```
status VARCHAR(20) DEFAULT 'OPEN',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

UNIQUE KEY (customer_id, order_no, order_type, version_no),
INDEX idx_status (status),
FOREIGN KEY (customer_id) REFERENCES m_customer(id)
);
```

主要フィールド

フィールド名	型	説明
order_type	VARCHAR(20)	受注タイプ (FIRM/FORECAST)
status	VARCHAR(20)	ステータス (OPEN/SUPERSEDED/CLOSED/CANCELED)
freeze_from	DATE	凍結開始日 (この日以降の変更を禁止)

2. t_order_line (受注明細)

受注の製品別・納期別の詳細情報を管理。スケジューリングエンジンはこのテーブルを参照して生産計画を立てる。

テーブル構造

```
CREATE TABLE t_order_line (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id BIGINT NOT NULL,
  line_no INT NOT NULL,
  product_id BIGINT,
  product_code VARCHAR(50) NOT NULL,
  order_type VARCHAR(20),
  customer_order_no VARCHAR(50),
  quantity DECIMAL(14,3) NOT NULL,
  actual_shipment_qty DECIMAL(14,3) DEFAULT 0,
  due_date DATE NOT NULL,
  plant_code VARCHAR(20),
  ship_to_code VARCHAR(40),
  remark VARCHAR(200),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  UNIQUE KEY (order_id, line_no),
  INDEX idx_due_date (due_date),
  INDEX idx_product_code (product_code),
  FOREIGN KEY (order_id) REFERENCES t_order(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES m_product(id) ON DELETE SET NULL
);
```

主要フィールド

フィールド名	型	説明
due_date	DATE	納期（スケジューリングの基準日）
quantity	DECIMAL(14,3)	数量
product_code	VARCHAR(50)	製品コード
customer_order_no	VARCHAR(50)	顧客発注番号（リーデン: 発注番号、ティエラ: 注文番号）
actual_shipment_qty	DECIMAL(14,3)	出荷実績数

3. 受注取込ステージング（顧客別Raw）

CSVファイルから取り込んだ生データを顧客ごとに最適化した形式で保存。解析前の状態を保持し、トラブルシューティングに使用。

設計方針: 顧客ごとにCSVフォーマットが大きく異なるため、顧客別にRawテーブルを分離。変換後のStgOrderDailyで統一フォーマットに変換。

3-1. stg_order_raw_tiera（ティエラ専用）

ティエラのCSVフォーマット（B17内示/Y55確定）に最適化。

```
CREATE TABLE stg_order_raw_tiera (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  customer_code VARCHAR(20) NOT NULL,  
  order_type VARCHAR(20) NOT NULL,  -- 'FIRM' or 'FORECAST'  
  source_system VARCHAR(50),  
  source_file VARCHAR(200),  
  source_row_no INT NOT NULL,  
  
  -- ティエラ固有フィールド  
  data_type VARCHAR(10),  -- 'B17' or 'Y55'  
  product_code VARCHAR(50),  -- 図番  
  due_date DATE,  
  quantity DECIMAL(14,3),  
  product_name VARCHAR(200),  -- 納品書用品名  
  product_name_kana VARCHAR(200),  -- 納品書用品名カナ  
  c_table_no VARCHAR(50),  -- C表No/不良通知No  
  
  raw_payload JSON NOT NULL,  
  parse_status VARCHAR(20) DEFAULT 'PENDING',  
  error_message TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  
  INDEX idx_customer_code (customer_code),  
  INDEX idx_parse_status (parse_status),  
)
```

```
INDEX idx_source_file (source_file)
);
```

3-2. stg_order_raw_rieden (リーデン専用)

リーデンのCSVフォーマット（発注コード=509）に最適化。シンプルな確定注文形式。

```
CREATE TABLE stg_order_raw_rieden (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_code VARCHAR(20) NOT NULL,
  order_type VARCHAR(20) NOT NULL, -- 'FIRM' or 'FORECAST'
  source_system VARCHAR(50),
  source_file VARCHAR(200),
  source_row_no INT NOT NULL,

  -- リーデン固有フィールド
  order_code VARCHAR(10), -- '509'
  product_code VARCHAR(50),
  due_date DATE,
  quantity DECIMAL(14,3),

  raw_payload JSON NOT NULL,
  parse_status VARCHAR(20) DEFAULT 'PENDING',
  error_message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  INDEX idx_customer_code (customer_code),
  INDEX idx_parse_status (parse_status),
  INDEX idx_source_file (source_file)
);
```

3-3. stg_order_raw_kubota (クボタ専用)

クボタのCSVフォーマット（36内示/45・47確定）に最適化。内示は横展開データ（3ヶ月×31日）を保存。

```
CREATE TABLE stg_order_raw_kubota (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_code VARCHAR(20) NOT NULL,
  order_type VARCHAR(20) NOT NULL, -- 'FIRM' or 'FORECAST'
  source_system VARCHAR(50),
  source_file VARCHAR(200),
  source_row_no INT NOT NULL,

  -- クボタ固有フィールド
  data_no VARCHAR(10), -- '36' or '45' or '47'
  record_type VARCHAR(10), -- 'V2' or 'V3'（内示のみ）
  product_code VARCHAR(50), -- 品番
  inspection_type VARCHAR(10), -- 検査区分（N, NS, TS, $ など）
);
```

```

product_name VARCHAR(200), -- 品名

-- 内示用 (36番) - 横展開データ
start_month VARCHAR(10), -- スタート月度 (例: 2512)
date_headers JSON, -- V2行の日付ヘッダー配列 ["51201", "51202", ...]
quantities JSON, -- V3行の数量配列 [72, 56, 64, ...]

-- 確定用 (45/47番)
delivery_date DATE, -- 納入指示日
quantity DECIMAL(14,3), -- 納入指示数
order_no VARCHAR(50), -- 注番

raw_payload JSON NOT NULL,
parse_status VARCHAR(20) DEFAULT 'PENDING',
error_message TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

INDEX idx_customer_code (customer_code),
INDEX idx_parse_status (parse_status),
INDEX idx_source_file (source_file),
INDEX idx_product_inspection (product_code, inspection_type)
);

```

3-4. stg_order_raw (旧モデル・非推奨)

⚠ **非推奨**: このテーブルは互換性のために残されていますが、新規顧客は顧客別Rawテーブルを使用してください。

汎用CSVフォーマット用のレガシーテーブル。将来的に削除予定。

```

CREATE TABLE stg_order_raw (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_code VARCHAR(20) NOT NULL,
  order_type VARCHAR(20) NOT NULL,
  source_system VARCHAR(50),
  source_file VARCHAR(200),
  source_row_no INT NOT NULL,
  record_token VARCHAR(100),

  -- 汎用フィールド
  due_date DATE,
  product_code VARCHAR(50),
  product_name VARCHAR(200),
  product_name_halfwidth VARCHAR(200),
  quantity DECIMAL(14,3),

  raw_payload JSON NOT NULL,
  parse_status VARCHAR(20) DEFAULT 'PENDING',
  error_message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  
INDEX idx_customer_code (customer_code),  
INDEX idx_parse_status (parse_status)  
);
```

データ構造の違い

顧客	CSVフォーマット	Raw保存形式	特徴
ティエラ	B17/Y55	1行=1明細	シンプル、縦持ちデータ
リーデン	発注コード=509	1行=1明細	シンプル、縦持ちデータ
クボタ（内示）	36番（V2+V3ペア）	V2+V3ペアで保存	複雑、横展開データ（3ヶ月×31日）
クボタ（確定）	45/47番	1行=1明細	シンプル、縦持ちデータ
クボタ（確定・特殊）	NVAN-2（47番）	1行=1明細	枚方特殊確定、専用エンドポイント
その他（旧）	汎用	1行=1明細	非推奨、新規は顧客別Rawを使用

解析ステータス（parse_status）

parse_status	説明
PENDING	未処理
PARSED	処理済
ERROR	エラー

4. stg_order_daily（受注取込ステージング：日別正規化）

顧客別Rawテーブル（stg_order_raw_tiera/rieden/kubota）を解析・正規化した**統一フォーマット**のデータ。

役割:

- 顧客ごとに異なるCSVフォーマットを統一フォーマットに変換
- クボタ内示の横展開データ（3ヶ月×31日）を縦持ちデータ（日別レコード）に展開
- 受注作成（Order/OrderLine）前の最終ステージング

テーブル構造

```
CREATE TABLE stg_order_daily (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  -- 元データへの参照（どれか一つのみNULLでない）  
  raw_id BIGINT, -- 旧モデル（非推奨）
```

```

raw_tiera_id BIGINT,
raw_rieden_id BIGINT,
raw_kubota_id BIGINT,

customer_id BIGINT,
order_type VARCHAR(20) NOT NULL,
version_no VARCHAR(20) DEFAULT 'v1',
product_code VARCHAR(50) NOT NULL,
product_name VARCHAR(200),
product_name_halfwidth VARCHAR(200),
due_date DATE NOT NULL,
quantity DECIMAL(14,3) NOT NULL,
plant_code VARCHAR(20),
ship_to_code VARCHAR(40),
source_system VARCHAR(50),
source_file VARCHAR(200),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

INDEX idx_due_date (due_date),
INDEX idx_customer_due_date (customer_id, due_date),
INDEX idx_product_code (product_code),
FOREIGN KEY (raw_id) REFERENCES stg_order_raw(id) ON DELETE CASCADE,
FOREIGN KEY (raw_tiera_id) REFERENCES stg_order_raw_tiera(id) ON DELETE
CASCADE,
FOREIGN KEY (raw_rieden_id) REFERENCES stg_order_raw_rieden(id) ON DELETE
CASCADE,
FOREIGN KEY (raw_kubota_id) REFERENCES stg_order_raw_kubota(id) ON DELETE
CASCADE,
FOREIGN KEY (customer_id) REFERENCES m_customer(id) ON DELETE SET NULL,

CONSTRAINT chk_raw_source CHECK (
    -- どれか一つだけがNOT NULL
    (raw_id IS NOT NULL AND raw_tiera_id IS NULL AND raw_rieden_id IS NULL AND
raw_kubota_id IS NULL) OR
    (raw_id IS NULL AND raw_tiera_id IS NOT NULL AND raw_rieden_id IS NULL AND
raw_kubota_id IS NULL) OR
    (raw_id IS NULL AND raw_tiera_id IS NULL AND raw_rieden_id IS NOT NULL AND
raw_kubota_id IS NULL) OR
    (raw_id IS NULL AND raw_tiera_id IS NULL AND raw_rieden_id IS NULL AND
raw_kubota_id IS NOT NULL)
)
);

```

データ変換例

クボタ内示（36番）の変換:

Raw（横展開）：

```

product_code: V053143521
inspection_type: N

```



```
record_type: V3
quantities: [72, 56, 64, 56, ...] ← 93個の要素 (3ヶ月×31日)
date_headers: ["51201", "51202", ...]
```

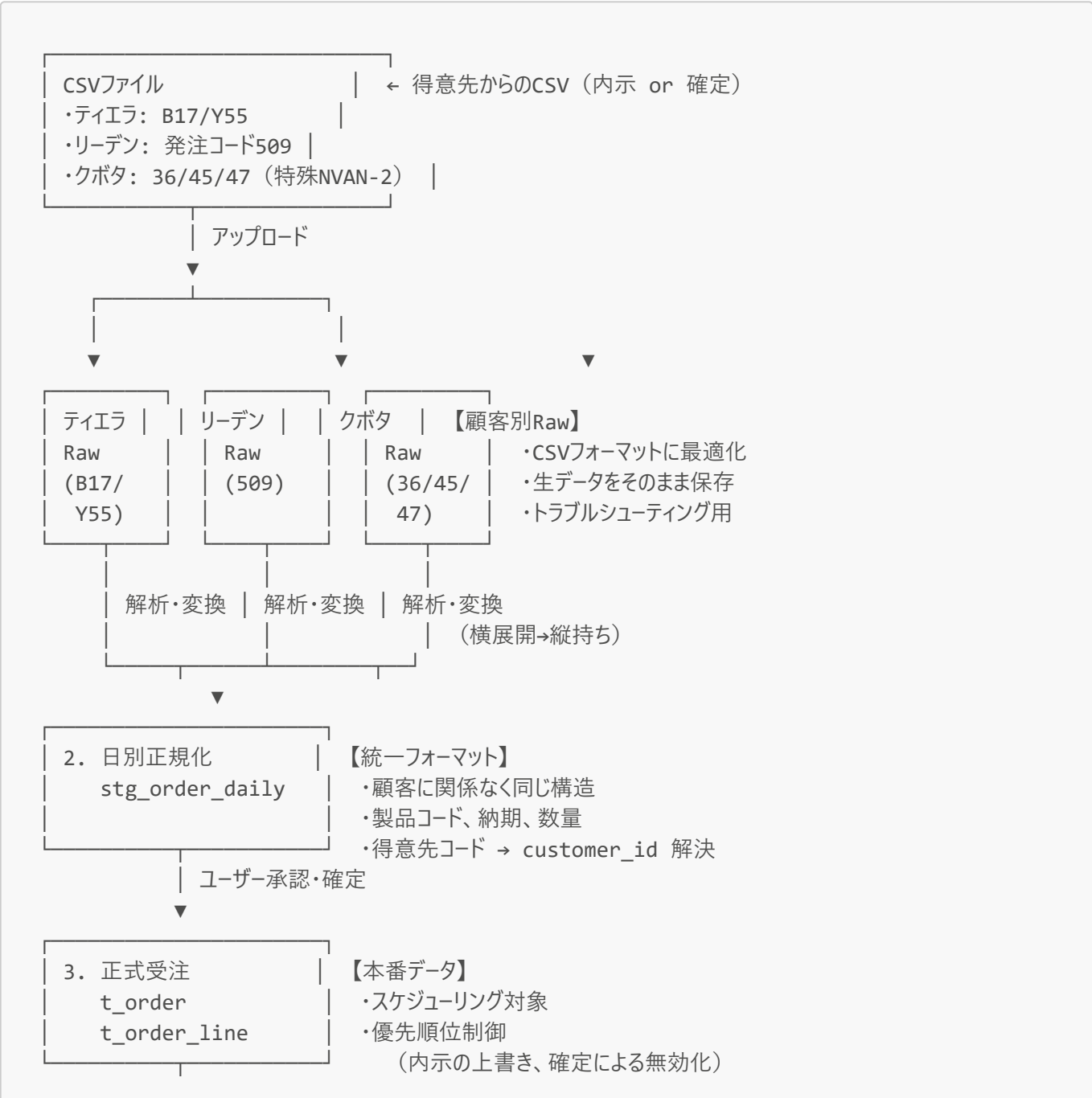
↓ 変換

Daily (縦持ち) :

```
product_code: V053143521, due_date: 2025-12-01, quantity: 72
product_code: V053143521, due_date: 2025-12-02, quantity: 56
product_code: V053143521, due_date: 2025-12-03, quantity: 64
...
```

データフロー

全体フロー (顧客別ステージング)



↓
▼
スケジューリングエンジン

- ・ライン選択 → 全顧客の製品表示
- ・顧客選択 → 特定顧客の製品表示

優先順位制御ロジック

1. 内示の更新（製品ごとの部分置換方式）

課題

内示は毎日更新される予測データだが、以下の特性がある：

- ・ 内示CSVには「今日より先の計画」のみ含まれる（開始日は製品ごとに異なる）
- ・ 内示の開始日より前の期間は、確定受注が入る予定の期間
- ・ 新しい内示で古い内示を完全置換すると、確定待ちの期間のデータが消える

解決策（製品ごとの部分置換）

新しい内示CSVがインポートされると、**製品ごとに、新しい内示の開始日以降のみ既存内示を無効化**する。

- ・ 新しい内示がカバーする範囲：置き換え
- ・ 新しい内示に含まれない近い未来：既存内示を残す（確定待ち）

処理フロー

1. 内示CSVアップロード
2. ステージングテーブルへ保存
3. ユーザーが「受注作成」実行
↓
4. 製品ごとの最小納期（開始日）を記録
例：製品A → 12/16、製品B → 12/20、製品C → 1/6
↓
5. 製品ごとに、開始日以降の既存OPEN内示を無効化（SUPERSEDED）
例：製品Aは 12/16以降の既存内示を無効化
→ 12/15の内示は残る（確定待ち）
↓
6. 新しい内示オーダを作成し、全ての明細を追加

実装コード

```
# pm_backend/apps/orders/core/services/csv_import.py:265-305
if order_type == 'FORECAST':
    # 製品ごとの最小納期を記録
    product_earliest_dates = {}
```

```
for daily in dailies:
    current_earliest = product_earliest_dates.get(daily.product_code)
    if current_earliest is None or daily.due_date < current_earliest:
        product_earliest_dates[daily.product_code] = daily.due_date

# 製品ごとに、開始日以降の既存内示を無効化
for product_code, earliest_date in product_earliest_dates.items():
    Order.objects.filter(
        customer_id=customer_id,
        order_type='FORECAST',
        status='OPEN',
        lines__product_code=product_code,
        lines__due_date__gte=earliest_date # 開始日以降のみ
    ).distinct().update(status='SUPERSEDED')

# 新しい内示明細を全て作成
for daily in dailies:
    OrderLine.objects.create(
        order=order,
        product_code=daily.product_code,
        quantity=daily.quantity,
        due_date=daily.due_date,
        ...
    )
```

結果

- 新しい内示がカバーする範囲（開始日以降）のみ置き換え
- 近い未来（開始日より前）の内示は残る
- 確定受注が来たら、その時点で内示を無効化（次のセクション参照）

実データ分析結果（ティエラ）

12月2日インポート時の実際のデータ：

製品コード	内示開始日	確定受注範囲	動作
3114047	12/16～	～12/16（予定）	12/16以降を置換、12/15は残る
3016161314	1/6～	～1/5（予定）	1/6以降を置換、それ以前は残る
4388286	12/22～	～12/21（予定）	12/22以降を置換、12/21以前は残る

規律: 内示の開始日 ÷ 確定受注の最終日 + 1日

2. 確定受注と内示の独立管理

設計方針（2026年1月変更）

確定受注インポート時は内示のステータスを変更しない。

- **内示（FORECAST）**：長期的な生産計画用の予測データ
- **確定受注（FIRM）**：短期的な確実な受注データ

両者は**独立して管理**され、画面では以下のように表示される：

項目	データソース	用途
内示	FORECAST（OPEN）	長期生産計画の参考
確定	FIRM（OPEN）	確実な生産計画
需要	内示 or 確定	生産計画の基準（確定優先）

処理フロー

1. 確定CSVアップロード

2. ステージングテーブルへ保存

3. ユーザーが「受注作成」実行

↓

4. 確定受注を作成

↓

5. 内示は変更しない（OPENのまま維持）

実装

```
# pm_backend/apps/orders/core/services/csv_import.py:376-379
# 確定インポート時は内示をSUPERSEDEDにしない
# 内示インポート時のみ、古い内示をSUPERSEDEDにする
# （確定と内示は別々に管理する）
pass
```

変更理由

旧仕様の問題点:

- 確定受注インポート時に内示をSUPERSEDEDに変更していた
- その後、新しい内示がインポートされないと、OPENの内示が存在しなくなる
- 画面で内示が表示されない問題が発生

新仕様の利点:

- 内示は内示インポート時のみ更新される（古い内示 → SUPERSEDED、新しい内示 → OPEN）
- 確定受注は独立して管理される
- 画面では両方が常に表示される（内示=長期計画、確定=短期確定）

3. スケジューリング時の優先順位

基本ルール

- 1. **status='OPEN'** の受注のみを使用
- 2. **確定（FIRM）を優先**
- 3. **確定がない製品・納期のみ、内示（FORECAST）を使用**

スケジューリング用データ取得

```
from orders.core.services.csv_import import CSVImportService

# スケジューリング用の受注明細を取得
order_lines = CSVImportService.get_active_order_lines_for_scheduling(
    customer_id=1,
    start_date=date(2025, 12, 1),
    end_date=date(2025, 12, 31)
)

# 結果は due_date, order_type でソート済み
# スケジューリングエンジンは確定を優先して処理
for line in order_lines:
    if line.order.order_type == 'FIRM':
        # 確定受注 → 必ず生産計画に含める
        schedule_production(line)
    else: # FORECAST
        # 内示 → 同じ製品・納期に確定がない場合のみ使用
        if not has_firm_order(line.product_code, line.due_date):
            schedule_tentative_production(line)
```

CSVインポート仕様

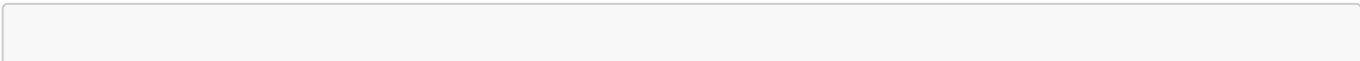
対応得意先

得意先コード	得意先名	内示	確定	フォーマット識別
000001	ティエラ	○	○	データ区分（B17/Y55）
000196	クボタ（堺・枚方）	○	○	工場選択 + データNO/No.（枚方特殊は専用エンドポイント）
000018	リーデン	-	○	発注コード（509）

サービス指向アーキテクチャ

得意先コードをキーにして、適切なインポートサービスを選択するファクトリパターンを採用。

ファイル構成



```
pm_backend/apps/orders/core/services/
├── __init__.py
├── README.md
├── base_import.py           # 共通処理（保存・正規化・自動登録）
├── csv_import.py           # デフォルト/汎用CSVインポート（ヘッダ列名）
├── tiera_naiji_import.py    # ティエラ内示
├── tiera_kakutei_import.py  # ティエラ確定
├── kubota_sakai_naiji_import.py
├── kubota_sakai_kakutei_import.py
├── kubota_hirakata_naiji_import.py
├── kubota_hirakata_kakutei_import.py
├── kubota_hirakata_special_kakutei_import.py
└── rieden_kakutei_import.py
```

インポート処理フロー

1. CSVファイルアップロード
2. 得意先コード + 受注タイプ + 工場指定で適切なサービスを選択（必要に応じてファイル名も参照）
 - 枚方特殊確定（NVAN-2）は専用エンドポイントで取り込み
3. 複数エンコーディングで自動デコード（CP932, Shift-JIS, UTF-8）
4. CSV解析、データ抽出
5. `stg_order_raw` / `stg_order_raw_*` へ生データ保存
6. 解析・正規化して `stg_order_daily` へ保存
7. 製品マスタ自動登録（BaseImportService 系サービスの場合）

デフォルトCSV形式（csv_import.py）

デフォルトサービスは、以下のヘッダーを持つCSVを想定します。

Column Name	必須	説明	形式
product_code	Yes	製品/部品コード	String
product_name	No	品名（全角）	String
product_name_halfwidth	No	品名（半角）	String
due_date	Yes	納期	YYYY-MM-DD or YYYY/MM/DD
quantity	Yes	受注数量	Number
plant_code	No	工場コード	String
ship_to_code	No	納入先コード	String
record_token	No	レコード識別子（任意）	String

補足: BaseImportService を使うサービスでは、`product_code` が `m_product` に存在しない場合、CSVの `product_name` / `product_name_halfwidth` を使って自動登録します（未指定の場合は `product_code` を品名として登録）。

クボタ 枚方特殊確定（NVAN-2）

- 対象: 得意先 000196 / 確定 (FIRM) / 枚方工場の特殊CSV
- 取り込みAPI: `POST /api/stg-order-raw/upload_hirakata_special/`
- 形式: ヘッダあり、列位置固定、`NO=47` の行のみ取り込み
- 列番号 (0-based) :
 - 0: NO
 - 3: 注番 (order_no)
 - 5: 品番 (product_code)
 - 10: 品名
 - 14: 検区
 - 23: 納期 (MMDD または MDD)
 - 24: 指示数
 - 26: 発行日 (YYMMDD)
- 受注番号: `FIRM-000196-HIRAKATA-sp-{issue_date}`
- 再インポート時は同一 `order_no` の明細を入れ替え (上書き)

CSV例

```
product_code,product_name,product_name_halfwidth,due_date,quantity,plant_code,ship_to_code
PROD001,製品名 1,Product Name 1,2025-01-15,100,PLANT01,SHIP01
PROD002,製品名 2,Product Name 2,2025-01-20,200,PLANT01,SHIP02
PROD003,製品名 3,Product Name 3,2025-02-10,150,PLANT02,SHIP01
```

サンプルファイル: `d:\pm\sample_order.csv`

製品マスタ自動登録

CSVに存在しない製品コードが含まれていた場合、自動的に製品マスタに登録:

```
product, created = Product.objects.get_or_create(
    product_code=raw.product_code,
    defaults={
        'product_name': product_name_for_master,
        'product_name_halfwidth': product_name_half_for_master,
        'category': 'PURCHASED', # デフォルトは購入品
        'unit': '個',
        'is_active': True,
        'is_final_product': True
    }
)
```

得意先専用インポートの作成

Step 1: サービスファイルを作成

新規ファイル: `pm_backend/apps/orders/core/services/[customer_name]_import.py`

例: `tiera_import.py`, `honda_import.py`

Step 2: インポートサービスを実装

```
# pm_backend/apps/orders/core/services/tiera_import.py
import csv
from datetime import datetime
from decimal import Decimal
from django.db import transaction
from orders.core.models import StgOrderRaw
from masters.models import Customer

class TieraNaijiImportService:
    """ティアエラ内示 (FORECAST) 用CSVインポート"""

    # Column index definitions (0-based)
    IDENTIFIER_COL = 0      # データ区分 = "B17"
    COL_PRODUCT_CODE = 6    # 図番
    COL_DELIVERY_DATE = 8   # 納期 (YYYYMMDD)
    COL_QUANTITY = 11       # 数量
    COL_PRODUCT_NAME_FULL = 12 # 品名 (全角) 列13
    COL_PRODUCT_NAME_HALF = 13 # 品名半角 列14

    def __init__(self):
        self.errors = []
        self.warnings = []

    def import_csv(self, file, customer_code, order_type, source_system='CSV'):
        """ティアエラ専用フォーマットを取り込み"""
        self.errors = []
        self.warnings = []

        try:
            # CP932 (Shift-JIS系) で読み込み
            file.seek(0)
            raw_data = file.read()

            # デコード試行
            encodings = ['cp932', 'shift-jis', 'utf-8-sig', 'utf-8']
            decoded_file = None
            for encoding in encodings:
                try:
                    decoded_file = raw_data.decode(encoding)
                    break
                except UnicodeDecodeError:
                    continue

            if decoded_file is None:
                return {
                    'success': False,
                    'message': 'Failed to decode CSV file',
                    'errors': ['Unsupported encoding']
                }
        except Exception as e:
            self.errors.append(str(e))
```



```
}

# CSVを解析（ヘッダ無し、列番号で抽出）
lines = decoded_file.splitlines()
csv_reader = csv.reader(lines)

raw_records = []
row_no = 0

for row in csv_reader:
    row_no += 1

    # ヘッダ/空行のスキップ
    if row_no == 1 or len(row) < max(self.COL_PRODUCT_CODE,
self.COL_DELIVERY_DATE, self.COL_QUANTITY):
        continue

    try:
        # 列番号で抽出
        product_code = row[self.COL_PRODUCT_CODE].strip()
        product_name_full = row[self.COL_PRODUCT_NAME_FULL].strip() if
len(row) > self.COL_PRODUCT_NAME_FULL else ''
        product_name_half = row[self.COL_PRODUCT_NAME_HALF].strip() if
len(row) > self.COL_PRODUCT_NAME_HALF else ''
        delivery_date_str = row[self.COL_DELIVERY_DATE].strip()
        quantity_str = row[self.COL_QUANTITY].strip()

        # 日付パース（YYYYMMDD）
        due_date = None
        if delivery_date_str and len(delivery_date_str) == 8:
            try:
                due_date = datetime.strptime(delivery_date_str,
'%Y%m%d').date()
            except ValueError:
                self.warnings.append(f"Row {row_no}: Invalid date
format: {delivery_date_str}")

        # 数量パース
        quantity = None
        if quantity_str:
            try:
                quantity = Decimal(quantity_str)
            except:
                self.warnings.append(f"Row {row_no}: Invalid quantity:
{quantity_str}")

        # 生データを作成
        raw_record = StgOrderRaw(
            customer_code=customer_code,
            order_type=order_type,
            source_system=source_system,
            source_file=file.name,
            source_row_no=row_no,
            record_token='',
```

```

        due_date=due_date,
        product_code=product_code,
        product_name=product_name_full,
        product_name_halfwidth=product_name_half,
        quantity=quantity,
        raw_payload={'row': row}, # 行全体を保存
        parse_status='PENDING'
    )
    raw_records.append(raw_record)

except Exception as e:
    self.errors.append(f"Row {row_no}: {str(e)}")

if not raw_records:
    return {
        'success': False,
        'message': 'No valid records found',
        'errors': self.errors
    }

# DB保存（標準処理に委譲）
# ...（デフォルトサービスと同様）

return {
    'success': True,
    'message': f'{len(raw_records)} 件を取り込み',
    'raw_count': len(raw_records),
    'errors': self.errors,
    'warnings': self.warnings
}

except Exception as e:
    return {
        'success': False,
        'message': f'取込失敗: {str(e)}',
        'errors': self.errors + [str(e)]
    }

```

Step 3: views.py でサービスを登録

`pm_backend/apps/orders/core/views.py` を編集し、得意先コードと受注タイプに応じたサービスを紐づけます。

```

def _get_import_service(self, customer_code, order_type, filename, factory=None):
    """得意先コード・受注タイプ・ファイル名でインポートサービスを選択"""
    from orders.core.services.csv_import import CSVImportService

    filename_lower = filename.lower()

    if customer_code == '000001':
        if order_type == 'FORECAST':

```

```

        from orders.core.services.tiera_naiji_import import
TieraNaijiImportService
        return TieraNaijiImportService()
    if order_type == 'FIRM':
        from orders.core.services.tiera_kakutei_import import
TieraKakuteiImportService
        return TieraKakuteiImportService()

    if customer_code == '000196':
        detected_factory = factory
        if not detected_factory:
            if '堺' in filename or 'sakai' in filename_lower:
                detected_factory = 'SAKAI'
            elif '枚方' in filename or 'hirakata' in filename_lower:
                detected_factory = 'HIRAKATA'
            if detected_factory == 'SAKAI' and order_type == 'FORECAST':
                from orders.core.services.kubota_sakai_naiji_import import
KubotaSakaiNaijiImportService
                return KubotaSakaiNaijiImportService()
            if detected_factory == 'SAKAI' and order_type == 'FIRM':
                from orders.core.services.kubota_sakai_kakutei_import import
KubotaSakaiKakuteiImportService
                return KubotaSakaiKakuteiImportService()
            if detected_factory == 'HIRAKATA' and order_type == 'FORECAST':
                from orders.core.services.kubota_hirakata_naiji_import import
KubotaHirakataNaijiImportService
                return KubotaHirakataNaijiImportService()
            if detected_factory == 'HIRAKATA' and order_type == 'FIRM':
                from orders.core.services.kubota_hirakata_kakutei_import import
KubotaHirakataKakuteiImportService
                return KubotaHirakataKakuteiImportService()

    if customer_code == '000018' and order_type == 'FIRM':
        from orders.core.services.rieden_kakutei_import import
RiedenKakuteiImportService
        return RiedenKakuteiImportService()

    # デフォルト
    return CSVImportService()

```

補足: 枚方特殊確定 (NVAN-2) は `upload_hirakata_special` を使用し、上記の `_get_import_service` では選択しない。

テスト

1. デフォルトサービスのテスト:

- ファイル: `d:\pm\sample_order.csv`
- 得意先: 任意
- 期待結果: 3件の取込成功

2. 得意先専用フォーマットのテスト:

- 得意先専用のインポートサービスを作成
- views.py に登録
- CSVをアップロード
- 正しくパースされることを確認

列番号指定と列名指定

- **列番号指定**（固定フォーマット向け・推奨）：
 - ヘッダ無し / 列位置固定のCSVで使用
 - 例: ティエラの列固定フォーマット
 - Excel出力のCSVに強い
- **列名指定**（可変フォーマット向け）：
 - ヘッダ付きのCSVで使用
 - 例: デフォルトサービス (`product_code`, `due_date`, `quantity`)
 - 柔軟だがヘッダ名の統一が必要

トラブルシューティング

問題: すべて ERROR になる

原因: CSVフォーマットが想定と一致していない

対処:

1. CSVフォーマットを確認
2. 必要なら得意先専用サービスを作成
3. 列名/列番号が一致しているか確認

問題: 文字化け/デコードエラー

原因: 未対応のエンコーディング

対処: `encodings` に追加:

```
encodings = ['cp932', 'shift-jis', 'utf-8-sig', 'utf-8', 'iso-2022-jp']
```

問題: 日付パースエラー

原因: 日付形式が一致していない

対処: 複数フォーマット対応を追加:

```
# Try multiple date formats
for fmt in ['%Y%m%d', '%Y-%m-%d', '%Y/%m/%d']:
    try:
        due_date = datetime.strptime(date_str, fmt).date()
```

```
        break
    except ValueError:
        continue
```

スケジューリング連携

スケジューリングエンジンへの入力

スケジューリングエンジンは、以下のクエリで受注明細を取得：

```
# status='OPEN' かつ 確定を優先
order_lines = OrderLine.objects.filter(
    order__status='OPEN'
).select_related('order', 'product').order_by(
    'due_date',
    'order__order_type' # FIRM が FORECAST より先
)
```

生産計画への反映

1. **確定受注（FIRM）**：必ず生産計画に含める
2. **内示（FORECAST）**：確定がない製品・納期のみ参考として使用
3. **無効化された内示（SUPERSEDED）**：完全に無視

数量集約（Aggregation）

背景

実際のCSVデータ（例：ティエラ_確定.csv）では、**同じ製品コード + 同じ納期で複数行（10行以上）が存在**することがあります。これは、異なる注文番号で同じ製品・納期の受注が入るためです。

要件

1. **個別情報の保持**: 各注文番号と明細情報は、後で参照する必要があるため、個別に保存
2. **スケジューリング用の集約**: 生産計画を立てる際は、製品コード + 納期で数量を合計して使用

実装方法

`get_active_order_lines_for_scheduling()` メソッドは、`aggregate` パラメータで動作を切り替え：

方法A: 集約モード（`aggregate=True`、デフォルト）

製品コード + 納期で数量を合計したデータを返す：

```
from orders.core.services.csv_import import CSVImportService
```

```
# スケジューリング用：集約データ取得
aggregated = CSVImportService.get_active_order_lines_for_scheduling(
    customer_id=1,
    aggregate=True # デフォルト
)

for item in aggregated:
    print(f"{item['product_code']}: {item['total_quantity']} ({item['line_count']}
    行を集約)")
# 例: 24201-36010: 150.000 (12行を集約)
```

返却データ構造：

```
{
    'customer_id': 1,
    'customer_code': '000001',
    'product_code': '24201-36010',
    'due_date': date(2025, 12, 10),
    'total_quantity': Decimal('150.000'), # 合計数量
    'order_type': 'FIRM',
    'line_count': 12 # 集約した行数
}
```

方法B: 個別モード (aggregate=False)

すべての注文明細を個別に返す（注文番号などの詳細情報を含む）：

```
# 個別情報参照用：すべての明細取得
individual = CSVImportService.get_active_order_lines_for_scheduling(
    product_code='24201-36010',
    aggregate=False
)

for line in individual:
    print(f"Order: {line.order.order_no}, Line: {line.line_no}, Qty:
    {line.quantity}")
# 例:
# Order: FIRM-000001-20251204120000, Line: 1, Qty: 10.000
# Order: FIRM-000001-20251204120000, Line: 2, Qty: 15.000
# ...
```

使用例

詳細は [scheduling_example.py](#) を参照。

逆算ロジック

納期 (`due_date`) から逆算 (バックワードスケジューリング) して、各工程の開始・終了時刻を計算。

詳細は [仕様書/scheduling_logic_v3.md](#) を参照。

API仕様

受注管理API

エンドポイント	メソッド	説明
<code>/api/orders/</code>	GET	受注一覧取得
<code>/api/orders/{id}/</code>	GET	受注詳細取得
<code>/api/order-lines/</code>	GET	受注明細一覧取得
<code>/api/stg-order-raw/upload_csv/</code>	POST	CSV受注インポート
<code>/api/stg-order-raw/upload_hirakata_special/</code>	POST	枚方特殊確定CSVインポート
<code>/api/stg-order-raw/create_orders/</code>	POST	ステージングから受注作成
<code>/api/stg-order-daily/</code>	GET	日別正規化データ取得

CSV受注インポートAPI

エンドポイント: `POST /api/stg-order-raw/upload_csv/`

リクエスト:

```
POST /api/stg-order-raw/upload_csv/
Content-Type: multipart/form-data

file: (CSVファイル)
customer_code: "000001"
order_type: "FORECAST" # or "FIRM"
source_system: "CSV"
```

レスポンス:

```
{
  "success": true,
  "message": "Imported 150 raw records, created 150 daily records",
  "raw_count": 150,
  "daily_count": 150,
  "errors": [],
  "warnings": []
}
```

枚方特殊確定CSVインポートAPI

エンドポイント: `POST /api/stg-order-raw/upload_hirakata_special/`

リクエスト:

```
POST /api/stg-order-raw/upload_hirakata_special/  
Content-Type: multipart/form-data  
  
file: (CSVファイル)  
customer_code: "000196"  
order_type: "FIRM"  
source_system: "CSV"
```

補足:

- `customer_code` は `000196` のみ受け付ける
- 受注番号は `FIRM-000196-HIRAKATA-sp-{issue_date}` 形式で作成

受注作成API

エンドポイント: `POST /api/stg-order-raw/create_orders/`

レスポンス:

```
{  
  "orders": 1,  
  "lines": 150,  
  "deleted_forecast_orders": 1  
}
```

注意事項

内示の削除について

- 内示オーダ自体は自動削除しない。明細を新オーダへ付け替えるため、既存オーダに明細が残らなければ空オーダが残ることがある（必要ならクリーンアップ要）。

確定受注の無効化について

- `000001/000196` は「該当製品の確定納期の最遅日まで」をSUPERSEDED。その他は「製品コード×納期の一致」でSUPERSEDED。

製品マスタについて

- BasImportService 系インポートは未登録品を自動登録する。`csv_import.py` は製品マスタ作成を行わないため、未登録品は `product_id` がNULLのまま受注明細に入る（必要なら顧客別サービスを用

意)。

スケジューリングエンジンの実装

- 現在は `get_active_order_lines_for_scheduling()` メソッドが基本的なフィルタリングのみ実装
- 実際のスケジューリングロジックは将来実装予定 (`scheduling_logic_v3.md` 参照)

関連ドキュメント

- [弊社生産特徴まとめ](#) - システム全体の特徴と生産構造
- [マスタテーブル定義](#) - マスタデータのテーブル定義
- [スケジューリングロジック](#) - 逆算ロジック、日跨ぎ計算
- [ER図](#) - データベース全体のER図

納入先コード処理仕様

概要

納入先コード (`ship_to_code`) は、同一得意先内で複数の納入先がある場合に、出荷先を識別するためのコードです。納入先ごとにリードタイムや配送条件が異なる場合があるため、以下の機能を提供します：

1. 納入先コードの自動検出・抽出
2. 納入先コード正規化処理
3. 納入先別リードタイム管理
4. 営業日計算による出荷日自動算出

データモデル

OrderLine (受注明細)

```
CREATE TABLE t_order_line (  
    ...  
    ship_to_code VARCHAR(40), -- 納入先コード  
    ...  
);
```

StgOrderDaily (ステージング)

```
CREATE TABLE stg_order_daily (  
    ...  
    ship_to_code VARCHAR(40), -- 納入先コード  
    ...  
);
```

ShipmentActual（出荷実績）

```
CREATE TABLE t_shipment_actual (  
    ...  
    ship_to_code VARCHAR(40), -- 納入先コード  
    ...  
    INDEX idx_ship_to_code (ship_to_code)  
);
```

納入先コードの検出と正規化

CSVからの自動検出

以下のキーワードで納入先コード列を自動検出：

カテゴリ	キーワード
納入先キーワード	納入、納入先、delivery
コードキーワード	コード、ｺｰﾄﾞ、code、cd

検出例：

- ☒ 「納入先コード」
- ☒ 「納入先ｺｰﾄﾞ」
- ☒ 「納入コード」
- ☒ 「delivery code」

正規化処理

```
def normalize_ship_to_code(value: str) -> str:  
    """納入先コードの正規化  
  
    処理内容：  
    1. Unicode正規化（NFKC） - 全角→半角変換  
    2. ハイフン・スペース除去  
    3. 数字のみの場合、6桁ゼロパディング  
  
    例：  
    "000010" → "000010"  
    "10" → "000010"  
    " 1 0 " → "000010"  
    "0-0-0-1-0" → "000010"  
    ""  
  
    if not value:  
        return ''  
  
    import unicodedata
```

```
normalized = unicodedata.normalize('NFKC', value)
normalized = normalized.replace('-', '').replace(' ', '').replace(' ', '')

if normalized.isdigit():
    normalized = normalized.zfill(6)

return normalized
```

納入先別リードタイム管理

設定方法

各インポートサービスで納入先別のリードタイム（営業日）を定義：

```
# 例：リーデン確定インポートサービス
SHIP_TO_LEAD_TIME_DAYS = {
    "000010": 2, # 納入先コード 000010 → 2営業日前出荷
    "000030": 2, # 納入先コード 000030 → 2営業日前出荷
}
```

出荷日計算ロジック

納期（`due_date`）から納入先別リードタイムを引いて、出荷日を算出：

```
def calculate_shipping_date(due_date: date, ship_to_code: str, calendar: Calendar)
-> date:
    """納入先コードに基づいて出荷日を計算

    Args:
        due_date: 納期
        ship_to_code: 納入先コード
        calendar: カレンダー（営業日管理）

    Returns:
        出荷日（営業日ベース）
    """
    normalized_code = normalize_ship_to_code(ship_to_code)
    lead_days = SHIP_TO_LEAD_TIME_DAYS.get(normalized_code, 0)

    if lead_days <= 0:
        return due_date

    # カレンダーの営業日を使って逆算
    return subtract_working_days(due_date, lead_days, calendar)
```

営業日計算

カレンダーテーブル

```
-- カレンドマスタ
CREATE TABLE m_calendar (
    id BIGINT PRIMARY KEY,
    calendar_code VARCHAR(20) UNIQUE,
    calendar_name VARCHAR(50),
    ...
);

-- カレンド日マスタ
CREATE TABLE m_calendar_day (
    id BIGINT PRIMARY KEY,
    calendar_id BIGINT,
    target_date DATE,
    is_working_day BOOLEAN, -- 営業日フラグ
    work_minutes INT,
    work_pattern_id BIGINT,
    ...
    UNIQUE KEY (calendar_id, target_date)
);
```

営業日逆算ロジック

```
def subtract_working_days(base_date: date, days: int, calendar: Calendar) -> date:
    """営業日を逆算

    Args:
        base_date: 基準日（納期）
        days: 逆算する営業日数
        calendar: カレンダー

    Returns:
        逆算した日付（出荷日）

    例：
        base_date = 2026-01-15（納期）
        days = 2（営業日）
        → 2026-01-13（出荷日）
        ※ 土日祝日をスキップ
    """
    if days <= 0:
        return base_date

    remaining = days
    current = base_date
```

```
while remaining > 0:
    current = current - timedelta(days=1)

    # カレンダから営業日判定
    cal_day = CalendarDay.objects.filter(
        calendar=calendar,
        target_date=current
    ).first()

    is_working = cal_day.is_working_day if cal_day else (current.weekday() <
5)

    if is_working:
        remaining -= 1

return current
```

営業日計算の例

納期: 2026-01-15 (水)
納入先コード: 000010
リードタイム: 2営業日

計算:

2026-01-15 (水) ← 納期
2026-01-14 (火) ← 1営業日前
2026-01-13 (月) ← 2営業日前 ← 出荷日

※ 土日 (1/11, 1/12) はスキップ

CSVインポート時の処理フロー

1. CSV解析時

```
# 1. 納入先コード列を検出
ship_to_col = find_ship_to_code_column(csv_headers)

# 2. CSVから納入先コードを抽出
for row in csv_rows:
    ship_to_code = row[ship_to_col] if ship_to_col else ''
    ship_to_code = normalize_ship_to_code(ship_to_code)

# 3. StgOrderRaw に保存
raw_record.raw_payload['ship_to_code'] = ship_to_code
```

2. StgOrderDaily 作成時

```
# StgOrderDaily に納入先コードを展開
daily_record = StgOrderDaily(
    ...
    ship_to_code=raw.raw_payload.get('ship_to_code', ''),
    ...
)
```

3. 受注作成時

```
# OrderLine に納入先コードを反映
order_line = OrderLine(
    ...
    ship_to_code=daily.ship_to_code,
    ...
)
```

4. 出荷実績作成時

```
# ShipmentActual に納入先コードと計算済み出荷日を記録
shipment = ShipmentActual(
    shipment_date=calculate_shipping_date(
        due_date=order_line.due_date,
        ship_to_code=order_line.ship_to_code,
        calendar=customer.calendar
    ),
    ship_to_code=order_line.ship_to_code,
    ...
)
```

得意先別の対応状況

得意先	納入先コード対応	リードタイム設定	備考
ティエラ	○	未設定	納入先コード列を自動検出
リーデン	○（予定）	○（予定）	000010/000030 は 2営業日
クボタ	△	未設定	今後対応予定

関連API

出荷実績検索（納入先フィルタ）

```
GET /api/shipment-actuals/?ship_to_code=000010
```

受注明細検索（納入先フィルタ）

```
GET /api/order-lines/?ship_to_code=000010
```

注意事項

1. 営業日計算にはカレンダーマスタが必須
- 得意先マスタに `calendar` を設定する必要がある
 - カレンダーが未設定の場合、週末判定（月～金を営業日）で代替
2. 納入先コードは任意項目
- NULL許容のため、納入先がない場合は空文字列
3. リードタイム設定の変更
- リードタイム変更時は、各インポートサービスの定数を更新
 - 既存データには影響しない（新規インポート時のみ適用）
4. 出荷日の自動計算タイミング
- CSVインポート時に自動計算
 - 手動修正も可能（ShipmentActualの編集）

変更履歴

日付	変更内容
2025-12-04	初版作成：受注管理テーブル定義、内示・確定の優先順位制御ロジック統合
2025-12-04	数量集約機能追加：同一製品・納期の複数注文明細を集約してスケジューリング用データ取得
2025-12-31	内示更新ロジック変更：完全置換方式から製品ごとの部分置換方式へ変更。ティエラ実データ分析結果を追加
2025-12-31	クボタ受注対応：顧客別Rawテーブル設計追加（ <code>stg_order_raw_tiera/kubota</code> ）。 <code>StgOrderDaily</code> で統一フォーマットに変換
2025-12-31	リーデン対応： <code>stg_order_raw_rieden</code> テーブル追加。 <code>StgOrderRaw</code> （旧）を非推奨として明記。すべての主要顧客が顧客別Rawモデルに移行完了

日付	変更内容
2026-01-07	重要変更: 確定受注インポート時の内示無効化ロジックを削除。確定と内示は独立管理に変更。確定インポート時に内示のステータスを変更しないことで、内示が常にOPENで残るようになり、画面表示の安定性が向上
2026-01-09	納入先コード処理仕様追加: 納入先コードの自動検出・正規化・リードタイム管理・営業日計算による出荷日自動算出の仕様を追加